MODULE 33 · DOMAIN 4 - OPERATING, MONITORING, SECURING

FM and agent economics

Where generative cost actually comes from, and why estimates are usually wrong.

MODULE 33 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

Organisations consistently spend 1.5 to 2 times their initial Bedrock estimate.

Five ways Bedrock charges you

MODE	SHAPE
On-demand	Per input and output token. No commitment, no ceiling
Batch inference	Asynchronous, and priced around 50 percent below on-demand
Provisioned Throughput	Reserved model units per hour, used or not. Module 25
Custom Model Import	Per five-minute window per active copy. Module 25
Marketplace endpoints	Provider software plus running SageMaker AI infrastructure

ON THE EXAM

Only the first two scale with usage. The other three have a floor you pay regardless, which is the whole theme of this module.

Model choice, priced

DECISION	COST EFFECT
A smaller model in the same family	Often an order of magnitude cheaper per token
A capable model for hard requests only	Intelligent Prompt Routing. Module 12
A distilled model	Module 11: teacher's quality at a student's price
A fine-tuned small model	Can beat a large general model on a narrow task, far cheaper
The largest model everywhere	The default, and usually the most expensive possible answer

ON THE EXAM

Rates differ by orders of magnitude between models. No amount of prompt trimming recovers a bad model choice.

Distillation economics

- Distillation costs a **one-off training run** and produces a permanently cheaper model. Module 11.
- So it is an **investment**, and it pays back at a volume you can calculate in advance.
- Divide the training cost by the per-request saving to get the break-even request count.
- Below that volume it is a loss. Above it, the saving compounds for the model's whole life.
- The same arithmetic applies to module 22's optimisation jobs, and to any fine-tune.

> ON THE EXAM
>
> Every one-off optimisation has a break-even volume. Calculating it takes a minute and settles the argument.

Token pricing, and its asymmetry

- Charged per input token and per output token, at **different rates**.
- **Output is typically several times dearer than input.** Often four or five times.
- So a verbose answer costs considerably more than a long prompt of the same length.
- **maxTokens and 'answer concisely' are cost controls**, not just formatting preferences.
- Rates vary enormously between models, so the model choice is a cost decision. Module 12.

> ON THE EXAM
>
> The asymmetry is the practical point. Trimming the system prompt helps; trimming the response helps several times more.

Prompt caching economics

- A repeated prompt **prefix** can be cached, and cached tokens are charged at a large discount.
- The saving scales with **how much of your prompt repeats**, and how often.
- Ideal shape: a long system prompt or document, reused across many short questions.
- Poor shape: every request is different content, so there is no shared prefix to cache.
- Put the stable part first. A cache only helps if the repeated text is at the start. Module 12.

ON THE EXAM

Caching rewards prompt structure. The same instructions placed after the variable content save nothing at all.

Cross-Region inference profiles

- A cross-Region profile routes to capacity across a geography, improving throughput. Module 16.
- **Profiles can differ in price**, with some routing options cheaper than in-Region inference.
- So the routing decision is a cost decision as well as an availability one.
- **Residency may forbid it entirely**, and that constraint outranks the saving. Module 08.
- Latency-tolerant workloads are where the trade is easiest to accept.

> ON THE EXAM
>
> The saving is real and it is bounded by module 08. A residency requirement is not negotiable against a percentage.

Batch inference

- Submit many prompts as a job, get responses back in S3, asynchronously.
- **Priced around 50 percent below on-demand** for selected models.
- **Zero quality trade-off.** It is the same model, answering the same prompts.
- It also runs under **separate quotas**, so it does not consume your real-time capacity. Module 25.
- The cost is development time: an asynchronous S3-based workflow instead of a synchronous call.

> ON THE EXAM
>
> Half price for the same output is the largest discount in this module, and it applies to any work nobody is waiting for.

Real-time or batch?

REAL-TIME, WHEN

Somebody is waiting

- A user expects a response now.
- The work is triggered by a person.
- Full on-demand token price.
- Shares the real-time quota.

BATCH, WHEN

Nobody is waiting

- Nightly summarisation or classification.
- Dataset labelling, offline evaluation.
- About half the token price.
- Separate quotas, so it relieves pressure.

THE TIEBREAKER

Module 20's question again, now with a 50 percent price difference attached to the answer.

In a small RAG system, the vector store floor can cost more than all the tokens combined.

The OpenSearch Serverless floor

- A Knowledge Base on OpenSearch Serverless requires a **minimum of two OpenSearch Compute Units**.
- **That floor applies at zero query traffic.** The index existing is the cost.
- Published figures put it in the region of **350 dollars a month**, before a single query.
- A published worked example: **345 of floor against 222 of total inference**, at 50,000 queries a month.
- Rates change, so price it yourself. The **shape** is what to remember.

> ON THE EXAM
>
> This is the cost that does not appear on the model pricing page, because it belongs to a different service entirely.

Vector store cost shapes

OPTION	COST SHAPE
OpenSearch Serverless, production	A continuous floor. The meter runs at zero traffic
OpenSearch Serverless, dev-test	Roughly half the floor. For non-production only
Newer collection types	Scale to zero after an idle period, removing the minimum
S3 Vectors	Quoted at up to 90 percent cheaper. Module 09
pgvector on Aurora Serverless	Cheaper again for small to medium sets, if you run Aurora

ON THE EXAM

Module 09 chose a store on capability. This is the same decision seen through the bill, and the two can disagree.

Module 09's choice, priced

- Module 26 said **hybrid search requires OpenSearch Serverless**. That is the capability argument.
- This module says OpenSearch Serverless carries the **highest floor**. That is the cost argument.
- **Those two pull in opposite directions**, and a real design has to resolve it.
- At high query volume the floor is negligible and hybrid search is worth it.
- At low volume the floor dominates, and semantic-only on a cheaper store may be the right trade.

> ON THE EXAM
>
> Scale decides it. The same architectural choice is obviously right at one volume and obviously wrong at another.

Ingestion and embedding cost

- Embedding is charged on **input tokens only**, because embeddings produce no output tokens.
- **Initial ingestion is usually cheap**, and a surprisingly small line in most estimates.
- **Re-indexing is the cost that repeats.** Changing embedding model means embedding everything again.
- Parsing with **Bedrock Data Automation** is charged per page, which matters on a large corpus.
- Module 10's parsing choice is therefore a cost decision as well as a quality one.

> ON THE EXAM
>
> One-off ingestion rarely breaks a budget. A weekly full re-index of a large corpus can.

What one RAG question costs

COMPONENT	TYPICAL SHARE
Embedding the question	Negligible. A couple of hundred input tokens
Vector store query	Small per query, on top of the fixed floor
Retrieved chunks as input tokens	Large. Often thousands of tokens per question
Generated answer as output tokens	Large, and at the dearer output rate
Reranking, if enabled	A model call scoring every candidate. Module 26
Guardrail assessment	Charged per assessment, on input and output

ON THE EXAM

Retrieved chunks are the biggest input line. numberOfResults from module 26 is therefore one of the most direct cost levers you have.

Building a generative cost estimate

- **1. The floor.** What does this cost at zero traffic? Vector store, provisioned capacity, endpoints.
- **2. The unit.** Tokenise a real request and a real response. Do not guess token counts.
- **3. The multiplier.** Model calls per user action. For an agent, measure it rather than assuming one.
- **4. The extras.** Guardrails, evaluation, logging, storage, transfer.
- **5. Multiply by volume, then add 50 percent**, because estimates run low.

> ON THE EXAM
>
> Step two is where most estimates go wrong: people estimate tokens from word counts instead of measuring with the tokenizer.

A generative cost model worth having

LINE	WHY IT BELONGS
Fixed floor per month	Stated separately. No prompt change reduces it
Cost per user action	The unit that stays comparable as volume moves
Model calls per action	The multiplier, measured for agents
Rate end dates	Any promotional price, with an owner
Extras line	Guardrails, evaluation, logging, storage, transfer
Reconciliation date	When you will check this against the actual bill

ON THE EXAM

The last row is the one that makes the model honest. A forecast nobody ever checks is a forecast nobody learns from.

Agent token amplification

- **One user question can consume around ten times the tokens you expected.**
- The agent plans, searches, reads, decides, and answers. Each step is a full model call.
- **You pay for input and output of every intermediate step**, including reasoning nobody sees.
- Conversation history is resent each step, so later steps are dearer than earlier ones. Module 16.
- Module 27's steps-per-task metric is therefore your cost metric as well as your quality metric.

> ON THE EXAM
>
> Estimating an agent from user question count is the single largest source of generative budget overrun.

Controlling agent economics

- **Cap the steps.** A step limit is a budget limit, from module 27.
- **Use a cheaper model for the loop**, and a capable one only where judgement is needed.
- Trim or summarise history rather than resending everything every step.
- Give the agent **fewer, better tools**. Tool sprawl means more deliberation per step.
- And ask module 27's question: does this need an agent, or is it a workflow?

ON THE EXAM

The cheapest agent optimisation is deciding you did not need an agent. The second cheapest is a step limit.

The costs that surprise people

- **Vector store minimums**, which belong to another service entirely.
- **Embedding inference** for every query, not only for ingestion.
- **Guardrails per assessment**, charged on input and on output separately.
- **Agent token amplification**, which multiplies every other line.
- Plus Data Automation per page, Flow transitions, AgentCore tools, CloudWatch, S3 and network transfer.

> ON THE EXAM
>
> These four are the named causes of the 1.5x to 2x overrun. None of them is a hidden fee; all of them are on a different page.

What Guardrails and evaluation cost

- **Guardrails are charged per assessment**, and an application assessing input and output pays twice.
- That scales with traffic, so it is a per-request cost like tokens, not a floor.
- **Automatic evaluation** charges the model inference used, with the scoring itself not separately charged.
- **Human evaluation** charges per completed human task, which is why it is used to calibrate, not to measure everything.
- Scheduled evaluation from module 30 is a recurring cost. Size the set deliberately.

> ON THE EXAM
>
> Safety and measurement are per-request costs. They are correct to pay and they belong in the estimate from the start.

Launch pricing has an end date

- **Promotional launch rates are common**, and they are published with an end date.
- A forecast built on one is a forecast that steps up on a known day, and nobody remembers.
- Put the date and an owner in the financial model, next to the rate.
- The same applies to preview pricing and extended-access rates.
- Model lifecycle changes too: a model retiring forces a move, possibly to a dearer one. Module 12.

> ON THE EXAM
>
> Any rate in a forecast needs a date attached. A rate without a date is an assumption pretending to be a number.

Prompt optimisation

- Bedrock offers a **prompt optimisation** feature, charged per thousand tokens as a one-off.
- It rewrites a prompt to be more efficient, and the one-off cost is recouped through input token savings.
- System prompts grow. Instructions get added and never removed, and nobody re-reads them.
- Every token in a system prompt is sent on **every single request**, forever.
- A manual review achieves much of the same thing, and costs an afternoon.

> ON THE EXAM
>
> A 2,000-token system prompt on a million requests is two billion input tokens. Reading it once is worth doing.

Marketplace and self-hosted models

- **A Marketplace endpoint is not token-priced.** Treating it as if it were produces a very wrong estimate.
- You pay **provider software charges plus continuously running SageMaker AI infrastructure**.
- That is an **hourly** cost, so it behaves like module 32's endpoint, not like a token bill.
- The same applies to any open model you host yourself on an endpoint.
- At high volume that can be cheaper. At low volume it is usually far worse.

> ON THE EXAM
>
> Tokens scale with use; instances do not. Crossing between the two pricing shapes changes which volumes are economical.

Tokens against hosting

- Token pricing has **no floor and no ceiling**. Hosting has a floor and a capacity limit.
- **At low volume, tokens win. At high sustained volume, hosting can win.** There is a crossover.
- Find it by dividing the hourly hosting cost by the token cost of your average request.
- Provisioned Throughput sits between the two: reserved capacity, priced by throughput. Module 25.
- Most teams never reach the crossover, and reason about it as though they have.

> ON THE EXAM
>
> The crossover is a real calculation with a real answer. Do it before arguing about which model is 'cheaper'.

Reconciling against the bill

- The **Cost and Usage Report** gives spend at model, usage-type and time grain.
- **Per-call token and cache data needs invocation logging enabled**, from module 30. Off by default.
- Reconcile the two. Where they disagree, your mental model of the system is wrong somewhere.
- Do it once after launch, then monthly. The first reconciliation usually finds something.
- Module 32's tagging applies here too: tag Provisioned Throughput and supporting resources.

> ON THE EXAM
>
> Nobody's first generative cost model is right. The reconciliation is how you find out which assumption was wrong.

Budget guardrails for generative workloads

- **Quotas are a budget control**, not just a capacity one. A TPM ceiling caps monthly spend.
- **maxTokens caps the dearest half of every request**, and a step limit caps an agent's whole trajectory.
- Rate limit per user or tenant, so one caller cannot consume the account's capacity. Module 30.
- AWS Budgets alerts on forecast, so you hear while the month is still running. Module 32.
- There is no automatic spend cap on token usage. The controls are the ones you build.

> ON THE EXAM
>
> Unlike an instance, a token bill has no natural ceiling. Everything that bounds it is something you configured.

Unit economics for a generative system

UNIT	WHY IT IS THE USEFUL ONE
Cost per question answered	Comparable across months as volume changes
Cost per resolved ticket	Ties spend to the outcome the system exists for
Cost per agent task completed	Exposes the step multiplier directly
Tokens per request	The leading indicator. Rises before the bill does
Fixed cost per month	The floor. Worth stating separately from the variable part

ON THE EXAM

Separating fixed from variable is the whole trick. A total hides a floor that no amount of prompt tuning will reduce.

Tenant economics

- In a multi-tenant product, some tenants cost far more to serve than others.
- **Long documents, long conversations and heavy agent use** concentrate cost in a few accounts.
- **requestMetadata** or application-side accounting is how you see it. Modules 16 and 30.
- A flat price per seat against variable token cost is a margin problem waiting to appear.
- Knowing cost per tenant lets you price, cap, or tier deliberately rather than discovering it.

> ON THE EXAM
>
> Generative features break flat-rate pricing quietly, because usage varies by orders of magnitude between customers.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Bill roughly double the estimate	Vector store floor, embedding per query, Guardrails, or agent steps
Cost barely falls when traffic does	You are paying a fixed floor, not a variable rate
Agent costs far more than expected	Token amplification. Measure steps per task
Small pilot costs as much as production	The floor dominates at low volume
Cost stepped up on a specific date	Launch pricing expired
Cannot attribute spend to a feature	No requestMetadata and no application-side accounting
Marketplace model far dearer than expected	It is hourly infrastructure, not tokens

ON THE EXAM

Rows two and four are the same fact seen twice. A floor is invisible until volume is low enough for it to dominate.

Estimate the floor before the tokens. At small scale, the floor is the answer.

A team runs an internal RAG pilot on a Bedrock Knowledge Base backed by OpenSearch Serverless. It serves about 400 questions a month across a 2,000-document corpus.

Finance queries the bill. Token costs are a few dollars. The line item they are asking about is several hundred dollars and does not move with usage.

What is happening, and what should they consider?

A Retrieved chunks are inflating input tokens. Reduce numberOfResults

B The embedding model is being charged per query. Cache the query embeddings

C The OpenSearch Serverless minimum applies at zero traffic. Consider S3 Vectors or a dev-test collection

D Provisioned Throughput was purchased by mistake. Delete it

◆ PAUSE AND DECIDE

What is happening, and what should they consider?

A Retrieved chunks are inflating input tokens. Reduce numberOfResults

A real lever, and the scenario states token costs are a few dollars. Halving a few dollars does not address several hundred.

B The embedding model is being charged per query. Cache the query embeddings

Query embedding is a couple of hundred input tokens. At 400 questions a month it is negligible, and it would vary with usage.

C The OpenSearch Serverless minimum applies at zero traffic. Consider S3 Vectors or a dev-test collection

A two-OCU minimum produces a continuous floor regardless of queries. That matches a large charge that does not move with usage.

D Provisioned Throughput was purchased by mistake. Delete it

Would also produce a fixed charge, and nothing suggests it was bought. The Knowledge Base's own store explains it directly.

THE SIGNAL

'Does not move with usage' is the diagnostic phrase. Only a fixed floor behaves that way, and the vector store is the floor here.

A media company summarises around 80,000 articles every night, writing the summaries to a database used by the morning editorial team. The job currently calls the Converse API synchronously in a loop, and finishes around 4am.

Token spend is the dominant cost. Editorial only reads the summaries after 7am.

What should they change?

A Move to Bedrock batch inference, writing results to S3

B Buy Provisioned Throughput sized to the nightly peak

C Switch to a smaller model to halve the token rate

D Enable prompt caching on the summarisation prompt

◆ PAUSE AND DECIDE

What should they change?

A Move to Bedrock batch inference, writing results to S3

Nobody reads the output for three hours after it completes. Batch is around 50 percent cheaper with no quality change, and uses separate quotas.

B Buy Provisioned Throughput sized to the nightly peak

Reserves capacity around the clock for a job running a few hours a night. You would pay for the idle twenty hours as well.

C Switch to a smaller model to halve the token rate

Might work and changes the output. Batch halves the price of the model they already validated, with no quality trade-off.

D Enable prompt caching on the summarisation prompt

Helps where a long prefix repeats. Each article is different content, so the cacheable portion is only the instruction.

THE SIGNAL

'Editorial only reads them after 7am' is the permission slip. Latency tolerance is what batch inference charges half price for.

> A support agent built on AgentCore was budgeted by estimating 12,000 user questions a month at roughly 3,000 tokens each. The first month's bill is close to ten times that estimate.
>
> Question volume was as forecast. No model or prompt changed after launch.

What is the most likely explanation?

A Output tokens are charged at a higher rate than input tokens

B Each question triggers many model calls, and every intermediate step is billed

C The estimate omitted the vector store floor

D Launch pricing on the model expired during the month

◆ PAUSE AND DECIDE

What is the most likely explanation?

A. Output tokens are charged at a higher rate than input tokens

True, and it is a factor of a few, not ten. It also would have been visible in any per-token estimate.

B. Each question triggers many model calls, and every intermediate step is billed

An agent plans, calls tools, reads results and decides again. One user question can consume around ten times the expected tokens, and history is resent each step.

C. The estimate omitted the vector store floor

A real omission in many estimates, and a fixed monthly figure. It does not scale a bill to ten times the forecast.

D. Launch pricing on the model expired during the month

Would produce a step change of a defined size, and the scenario says nothing changed. A rate change does not multiply by ten.

THE SIGNAL

The estimate treated one question as one model call. For an agent it is a trajectory, and every step in it is billed.

> A team builds an eighteen-month financial model for a new assistant, using the current published rate for their chosen model. The rate is a promotional launch price with a published end date four months away.
>
> The model is approved by finance on that basis and the project proceeds.

What is wrong, and what should be done?

A. Nothing. Published rates are the correct basis for a forecast

B. Use Provisioned Throughput instead, which fixes the rate for the term

C. Use the cheapest available model instead, to build in headroom

D. Model the step up at the end date, with an owner, and revisit the choice before it

◆ PAUSE AND DECIDE

What is wrong, and what should be done?

A. Nothing. Published rates are the correct basis for a forecast

The rate is published with an end date four months into an eighteen-month model. Fourteen of those months are priced wrongly.

B. Use Provisioned Throughput instead, which fixes the rate for the term

Commits to reserved capacity for an unlaunched workload with no measured usage, which module 25 warned against specifically.

C. Use the cheapest available model instead, to build in headroom

Chooses a model on price before quality is established. Module 16's selection criteria come first, then cost.

D. Model the step up at the end date, with an owner, and revisit the choice before it

The end date is known and published, so the increase is forecastable. An owner means somebody revisits the model choice before it lands.

THE SIGNAL

The error is not using the promotional rate. It is using it for fourteen months in which it will not apply.

What to take from this module

- **Estimate the floor before the tokens.** At small scale the floor can exceed all inference.
- **Batch inference is around half price**, with no quality trade-off, for work nobody waits on.
- **Agents multiply tokens roughly tenfold.** Every intermediate step is billed.
- **Output tokens cost several times input.** Trimming the answer beats trimming the prompt.
- **Any rate in a forecast needs a date.** Launch pricing expires.

> ON THE EXAM
>
> Module 34 opens the security run: identity and access for ML and AI workloads.